

Exercitiul 10 — Factory Method: cine decide ce clasa se naste

Cerința exercițiului 10 din modulul de design patterns.

Context

Ai un fisier de utilizatori in care fiecare linie incepe cu un tip:

```
STUDENT,Ana,Popescu,anul 2
PROFESOR,Ion,Ionescu,Matematica
STUDENT,Vlad,Marin,anul 1
```

Trebuie sa construiesti, pentru fiecare linie, obiectul potrivit — `Student` sau `Profesor` — si sa le tii pe toate intr-un singur sir de `Utilizator`.

Prima varianta la care se gandeste oricine e un `switch` pe `campuri[0]` care se termina in `new`. Merge, si e aceeaasi durere de la `ex1`: ca sa adaugi `ADMIN`, redeschizi o metoda care mergea.

Dar de data asta Strategy nu te salveaza. Ai incercat deja varianta cu polimorfism in `academy` (`UserService.CreateUser`) si nu a mers — pentru ca `new User()` fixeaza tipul inainte ca polimorfismul sa apuce sa spuna ceva. Reciteste Lectia 3 daca nu-ti e clar de ce.

Factory Method e raspunsul: daca `new Student(...)` nu poate fi ales polimorfic, ascunzi `new`-ul intr-un obiect care POATE fi ales polimorfic.

Contractele

```
public abstract class Utilizator
{
    public string Prenume { get; }
    public string Nume { get; }
    public abstract string Descriere();
}
```

```
public interface IFabricaUtilizator
{
    string Tip { get; }
    Utilizator Creeaza(string[] campuri);
}
```

`IncarcatorUtilizatori` primeste un sir de fabrici prin constructor si nu cunoaste nicio clasa concreta de utilizator.

Ce construisti

Clasa	Ce face
<code>Student</code>	camp in plus <code>An</code> ; <code>Descriere()</code> -> <code>[STUDENT] Ana Popescu - anul 2</code>
<code>Profesor</code>	camp in plus <code>Catedra</code> ; <code>Descriere()</code> -> <code>[PROFESOR] Ion Ionescu - Matematica</code>
<code>FabricaStudent</code>	<code>Tip = "STUDENT"</code> ; <code>Creeaza</code> intoarce un <code>Student</code> construit din campuri
<code>FabricaProfesor</code>	<code>Tip = "PROFESOR"</code> ; <code>Creeaza</code> intoarce un <code>Profesor</code>
<code>IncarcatorUtilizatori</code>	<code>Incarca(string[] linii)</code> -> <code>Utilizator[]</code> , folosind fabricile primite

Ordinea campurilor pe linie: `TIP, Prenume, Nume, Detaliu` .

Pasii (in ordinea asta — primul pas e o capcana intentionata)

1. **Simte durerea intai.** Scrie `Incarca` cu `switch (campuri[0])` si `new Student(...)` / `new Profesor(...)` direct. Ruleaza, vezi ca merge. Apoi raspunde in scris, pe o foaie: daca maine apare `ADMIN` , ce fisiere atingi? Si a doua intrebare, mai importanta: **de ce nu poti scapa de `switch` -ul asta cu o metoda virtual ?**
2. **Muta fiecare `new` in fabrica lui.** `FabricaStudent` are un singur `new Student(...)` si nimic altceva. La fel `FabricaProfesor` .
3. **Sterge `switch` -ul din incarcator.** `Incarca` parcurge fabricile primite si o intreaba pe fiecare daca `Tip` -ul ei se potriveste cu `campuri[0]` . Cand se potriveste, cere produsul si merge mai departe. Daca niciuna nu se potriveste, arunca `ArgumentException` cu mesaj in engleza.
4. **Verifica.** Cauta cuvantul `new` in `IncarcatorUtilizatori` . Daca apare vreun `Student` sau `Profesor` acolo, pasul 3 nu e terminat.

Cerinte

1. `Student` si `Profesor` cu `Descriere()` suprascris.
2. `FabricaStudent` si `FabricaProfesor` .
3. `IncarcatorUtilizatori` cu `Incarca(string[] linii)` .
4. `Testare10` (chemata din `Program.cs`): un sir cu cele 3 linii din context, incarcat printr-un incarcator caruia ii dai cele doua fabrici. Afiseaza `Descriere()` pentru fiecare utilizator incarcat.
5. Demonstreaza cazul de eroare: mai adauga o linie cu un tip inexistent (`SECRETAR,...`), prinde `ArgumentException` si afiseaza mesajul.

Constrangeri

- `IncarcatorUtilizatori` NU are voie sa contina `new Student` , `new Profesor` , si niciun `switch` / `if` pe tipul de utilizator. Singura comparatie permisa e intre doua siruri de caractere: `Tip` -ul fabricii si `campuri[0]` .
- Fara `is` , `as` , `GetType()` . Utilizatorii se folosesc DOAR prin `Descriere()` .
- Fara LINQ, `List` , `Dictionary` . Siruri simple, ca pana acum.
- Mesajele de eroare in engleza.

Cum rulezi

```
dotnet run
```

Gata cand

Adaugi un tip nou de utilizator — `ADMIN, Alex, Grozavu, IT` — scriind **doar** o clasa `Admin` si o clasa `FabricaAdmin`, si trecand fabrica noua in sirul din `Testare10`. Daca a trebuit sa modifici o singura linie din `IncarcatorUtilizatori`, pattern-ul nu e complet.

Bonus

- **Fabrica cu validare proprie.** Fa `FabricaStudent.Creeaza` sa arunce `ArgumentException` daca linia nu are exact 4 campuri. Observa unde a ajuns validarea: langa `new` -ul pe care il apara, nu imprastiata prin incarcator.
- **Intrebare de gandit, pentru academy**. In `UserService.CreateUser` nu primesti o linie de text, ci un `UserCreateRequest` — si ai deja `StudentCreateRequest`, `TeacherCreateRequest`, `AdminCreateRequest`. Ce joaca rolul lui `campuri[0]` acolo, daca nu ai voie sa folosesti `as`? (Indiciu: ce ar putea sa aiba fiecare request, ca sa spuna singur cine e?)



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro